XML
DTD
XML Schema

Unit 4

XML

- eXtensible Markup Language
- Custom-defined tags
- Designed to store and transport data over the web
- Accessible from the web - not rendered as a web-page
    - https://www.w3schools.com/xml/plant_catalog.xml

Sample XML

```
<?xml version="1.0"?>
<customers>
   <customer id="55000">
      <name>Charter Group</name>
      <address>
         <street>100 Main</street>
         <city>Framingham</city>
         <state>MA</state>
         <zip>01701</zip>
      </address>
      <address>
         <street>720 Prospect</street>
         <city>Framingham</city>
         <state>MA</state>
         <zip>01701</zip>
      </address>
      <address>
         <street>120 Ridge</street>
         <state>MA</state>
         <zip>01760</zip>
      </address>
   </customer>
</customers>
```

Courtesy: IBM docs

Sample XML 2

```xml
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book category="cooking">
    <title lang="en">Everyday Italian</title>
    <author>Giada De Laurentiis</author>
    <year>2005</year>
    <price>30.00</price>
  </book>
  <book category="children">
    <title lang="en">Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
    <price>29.99</price>
  </book>
  <book category="web">
    <title lang="en">Learning XML</title>
    <author>Erik T. Ray</author>
    <year>2003</year>
    <price>39.95</price>
  </book>
</bookstore>
```

Courtesy: W3 school docs

HTML vs XML

- Purpose (Display versus Storage)
- Case-sensitivity
- User-defined tags
- Closing tags
- Whitespace (Indentation)
- Similarities with HTML
- Standard Generalized Markup Language

XML Structure

- Version
- Root
- Nested Child elements
- Attributes (values must be quoted)
- White space is preserved
- Defined by DTD and XML Schema

Class Exercises

- Design an XML for a list of users (name, email, etc) who have registered for a workshop

- Design an XML for a family tree representing 3 generations of people

- Design an XML to represent a conference schedule - sessions, speakers, time slots, locations

Why send and store data as XML ?

Validation in XML - DTD/ XML Schema

- Grammar for XML documents
- Describes and defines the structure of an XML document
- Constraints the custom tags
- DTD - Document Type Definition (not in XML syntax)
- XML Schema - written in XML (more powerful than DTD with support for many datatypes)

XML Validation

- **Data Integrity** - to avoid missing XML elements, wrong structure of data
- **Interoperability -** between tools , transfer of data
- **Security -** to prevent Xml Injection attacks
- **Error Detection** - detect errors in structure of data before processing

XXE Vulnerability

- XML eXternal Entity
- Attackers embed DTD in XML to get access to files

```
<?xml version="1.0"?>
<!DOCTYPE root [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<gene><name>&xxe;</name></gene>
```

DTD Internal Validation

```
<?xml version="1.0"?>
<!DOCTYPE note [
  <!ELEMENT note (to, from, heading, body)>
  <!ELEMENT to (#PCDATA)>
  <!ELEMENT from (#PCDATA)>
  <!ELEMENT heading (#PCDATA)>
  <!ELEMENT body (#PCDATA)>
]>
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

DTD External Validation

note.dtd

```
<!ELEMENT note (to, from, heading, body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
```

Reference in xml

```
<?xml version="1.0"?>
<!DOCTYPE note SYSTEM "note.dtd">
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

Sample XML/DTD/XML Schema

XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<student>
    <name>Nithya</name>
    <rollno>208321</rollno>
    <email>nithya@sun.com</email>
</student>
```

DTD:

```
 <!ELEMENT student (name, rollno, email)>
 <!ELEMENT name (#PCDATA)>
 <!ELEMENT rollno (#PCDATA)>
 <!ELEMENT email (#PCDATA)>
```

XML Schema :

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="student">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="name" type="xs:string"/>
        <xs:element name="rollno" type="xs:integer"/>
        <xs:element name="email" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

DTD Exercises

- Create a DTD for the following XML :

```
<book>
    <title>XML Basics</title>
    <author>John Doe</author>
    <price>450</price>
</book>
```

- Create a DTD for a student, with name, roll no, email address (optional), department.
  - `<!ELEMENT student (name, rollno, email?, department)>`
- Create a DTD for a <class> with multiple students (at least one student required)
  - `<!ELEMENT class (student+)>`
- Create a DTD for payment , where the payment can be cash or card
  - `<!ELEMENT payment (cash | card)>`

DTD Attributes

<!ATTLIST element-name attribute-name attribute-type default-value>

gene.dtd

```
<!ELEMENT gene (name, sequence?, modification*)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT sequence (#PCDATA)>
<!ELEMENT modification (type, position)>

<!ATTLIST gene
    id ID #REQUIRED
    species (human|mouse|yeast) "human"
    chromosome NMTOKEN #IMPLIED
    strand (plus|minus) "plus">

<!ATTLIST modification
    type (methylation|acetylation|phosphorylation) #REQUIRED
    position CDATA #REQUIRED>
```

gene.xml

```
<?xml version="1.0"?>
<!DOCTYPE gene SYSTEM "gene.dtd">
<gene id="TP53-001" species="human" chromosome="17" strand="plus">
  <name>Tumor Protein P53</name>
  <sequence>ATGCCG...</sequence>
  <modification type="methylation" position="chr17:7661779"/>
  <modification type="acetylation" position="chr17:7664200"/>
</gene>
```

Attribute Types

Type	Description	Example
CDATA	Character data (text)	<!ATTLIST img src CDATA #REQUIRED>
ID	Unique identifier	<!ATTLIST book id ID #REQUIRED>
IDREF	References an ID	<!ATTLIST chapter book IDREF #REQUIRED>
IDREFS	Space-separated IDREFs	<!ATTLIST index chapters IDREFS #IMPLIED>
ENTITY	Predeclared entity	<!ATTLIST image format ENTITY #FIXED "gif">
ENUMERATION	Fixed list:(value1 | value2)	<!ATTLIST shape type (circle|square) "circle">
NMTOKEN	Name token (no spaces)	<!ATTLIST token key NMTOKEN #REQUIRED>
NMTOKENS	Space-separated NMTOKENs	<!ATTLIST keys multiple NMTOKENS #IMPLIED>

Sample XML Schema

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           elementFormDefault="qualified">

  <!-- Root book element with required children -->
  <xs:element name="book">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="title" type="xs:string"/>
        <xs:element name="author" type="xs:string"/>
        <xs:element name="price" type="xs:decimal"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

XML Namespaces

- Distinguish between elements of different vocabularies
- Prevents naming conflicts when combining XML elements

```
<table>
  <name>Coffee Table</name>
  <width>80</width>
</table>
<table>
  <tr><td>Apples</td></tr>
</table>
```

HTML table or furniture table?

```
<h:html xmlns:h="http://www.w3.org/TR/html4/">
  <h:table>
    <h:tr><h:td>Apples</h:td></h:tr>
  </h:table>
</h:html>

<f:furniture xmlns:f="http://example.com/furniture">
  <f:table>
    <f:name>African Coffee Table</f:name>
    <f:width>80</f:width>
  </f:table>
</f:furniture>
```

Namespaces come to the rescue!

XMLSchema Built-In Datatypes

- xs:string
- xs:decimal
- xs:integer
- xs:boolean
- xs:date
- xs:time
- xs:float

https://www.liquid-technologies.com/Reference/XmlStudio/XsdEditorNotation_BuiltInXsdTypes.html

User defined datatypes - SimpleType and ComplexType

xs:simpleType

- No Child Elements
- No Attributes
- No sequence/all/choice
- For text/value

xs:complexType

- Can have Child Elements
- Can have Attributes
- Can have sequence/all/choice
- No value restrictions

SimpleType

Define ageType in schema:

```
<xs:simpleType name="ageType">
  <xs:restriction base="xs:int">
    <xs:minInclusive value="0"/>
    <xs:maxInclusive value="120"/>
  </xs:restriction>
</xs:simpleType>
```

Bind the element Age to the type ageType in schema:

```
<xs:element name="Age" type=“ageType"/>
```

In XML :

```
<Age>25</Age>
```

ComplexType

```
<xs:complexType name="StudentType">
  <xs:sequence>
    <xs:element name="name" type="xs:string"/>
    <xs:element name="age" type="ageType"/>
    <xs:element name="email" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

Bind the element Age to the type ageType in schema:

```
<xs:element name="student" type="StudentType"/>
```

In XML :

```
<student>
  <name>Alice</name>
  <age>22</age>
  <email>alice@gmail.com</email>
</student>
```

XSD Attributes

```
<xs:complexType name="bookType">
  <xs:attribute name="isbn" type="xs:string" use="required"/>
  <xs:attribute name="published" type="xs:date" use="optional"/>
</xs:complexType>
```

XML instance

```
<book xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:type="bookType"
      isbn="978-0-123456-78-9"
      published="2025-01-15">
 </book>
```

XML Schema exercises

- Create an XML schema for the following XML :

```
<book>
    <title>XML Basics</title>
    <author>John Doe</author>
    <price>450</price>
</book>
```

- Create an XML schema for a student, with name, roll no, email address (optional), department.

- Create an XML schema for a <class> with multiple students (at least one student required)

- Create an XML Schema for payment , where the payment can be cash or card

XML Schema examples

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="book">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="title" type="xs:string"/>
        <xs:element name="author" type="xs:string"/>
        <xs:element name="price" type="xs:decimal"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

```
<xs:element name="student">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="name" type="xs:string"/>
      <xs:element name="rollno" type="xs:string"/>
      <xs:element name="email" type="xs:string" minOccurs="0"/>
      <xs:element name="department" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

XML Schema examples

```
<xs:element name="class">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="student" type="studentType" minOccurs="1" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
<xs:complexType name="studentType">
  <xs:sequence>
    <xs:element name="name" type="xs:string"/>
    <xs:element name="rollno" type="xs:string"/>
    <xs:element name="email" type="xs:string" minOccurs="0"/>
    <xs:element name="department" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

```
<xs:element name="payment">
  <xs:complexType>
    <xs:choice>
      <xs:element name="cash">
        <xs:complexType>
          <xs:attribute name="amount" type="xs:decimal" use="required"/>
        </xs:complexType>
      </xs:element>
      <xs:element name="card">
        <xs:complexType>
          <xs:sequence>
            <xs:element name="number" type="xs:string"/>
            <xs:element name="expiry" type="xs:string"/>
          </xs:sequence>
        </xs:complexType>
      </xs:element>
    </xs:choice>
  </xs:complexType>
```